

OSSP-PRACTICAL-WEEK2

Task-1:

Develop a C program that uses the system calls `open()`, `read()`, `write()`, and `close()` to copy the contents of one file to another. Explain how control transitions between user space and kernel space during execution.

Source code:

```
GNU nano 7.2                                filecopy.c *
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>

int main()
{
    int source, destination;
    char buffer[1024];
    ssize_t bytesRead, bytesWritten;

    source = open("source.txt", O_RDONLY);

    if (source < 0)
    {
        perror("Error opening source file");
        return 1;
    }

    destination = open("destination.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (destination < 0)
    {
        perror("Error opening destination file");
        close(source);
        return 1;
    }

    while ((bytesRead = read(source, buffer, sizeof(buffer))) > 0)
    {
        bytesWritten = write(destination, buffer, bytesRead);

        if (bytesWritten < 0)
        {
            perror("Error writing to destination file");
            close(source);
            close(destination);
        }
    }
}
```

```

while ((bytesRead = read(source, buffer, sizeof(buffer))) > 0)
{
    bytesWritten = write(destination, buffer, bytesRead);

    if (bytesWritten < 0)
    {
        perror("Error writing to destination file");
        close(source);
        close(destination);
        return 1;
    }
}

if (bytesRead < 0)
{
    perror("Error reading source file");
}

close(source);
close(destination);

printf("File copied successfully.\n");

return 0;
}

```

Output:

```

sairam0806@LAPTOP-M6NQUIAP:/mnt/c/Users/SAIRAM0806$ ./filecopy
File copied successfully.
sairam0806@LAPTOP-M6NQUIAP:/mnt/c/Users/SAIRAM0806$ cat destination.txt
This is the source file.
This file will be copied using system calls.
This is a sample file for system call demonstration.

```

Observation:

I learned how Linux system calls such as `open()`, `read()`, `write()`, and `close()` allow a C program to interact with files. I also understood that these operations involve communication between user space and the Linux kernel, where the kernel provides the required file-handling services.

Task2:

Use the strace utility to trace all system calls generated by the command `cat sample.txt`. Analyze the sequence of system calls and identify the kernel services involved.

Source code:

```
GNU nano 7.2          sample.txt *  
Hello, this is a sample file.  
This file is used to demonstrate strace.
```

Output:

```
openat(AT_FDCWD, "sample.txt", O_RDONLY) = 3  
read(3, "Hello, this is a sample file.\nTh"... , 131072) = 72  
write(1, "Hello, this is a sample file.\nTh"... , 72Hello, this is a sample file  
This file is used to demonstrate strace.  
  
) = 72  
read(3, "", 131072)          = 0  
close(3)                     = 0  
close(1)                     = 0  
close(2)                     = 0  
exit_group(0)                 = ?  
+++ exited with 0 +++  
sairam0806@LAPTOP-M6NQUIAP:/mnt/c/Users/SAIRAM0806$
```

Observation:

I learned how the strace utility can be used to observe the system calls made by a Linux command. I understood how commands such as cat interact with the Linux kernel through system calls to open, read, display, and close a file.